

Mausam Technical Brief

Everything behind the personalized homepage — what each piece of the stack is, why it was chosen over the obvious alternative, how a request actually travels, and which numbers on screen are real. Written from the code, not from the plan.

4 RUNTIME DEPS	2 LIVE FEEDS	135 kB MAIN JS, GZIP
444 × 2 I18N KEYS	28/28 SKIES PASS AA	8 BUILD CHECKS

01

What this is

Problem statement **SIH26076** asks for a personalized homepage for Mausam, IMD's public weather app. Today every user sees the same generic screen.

The brief names eight user types and the data each one needs. Our build reflows the homepage around one to three of those personas, chosen at first launch and editable afterwards, and orders the cards by a rule that can be read out loud.

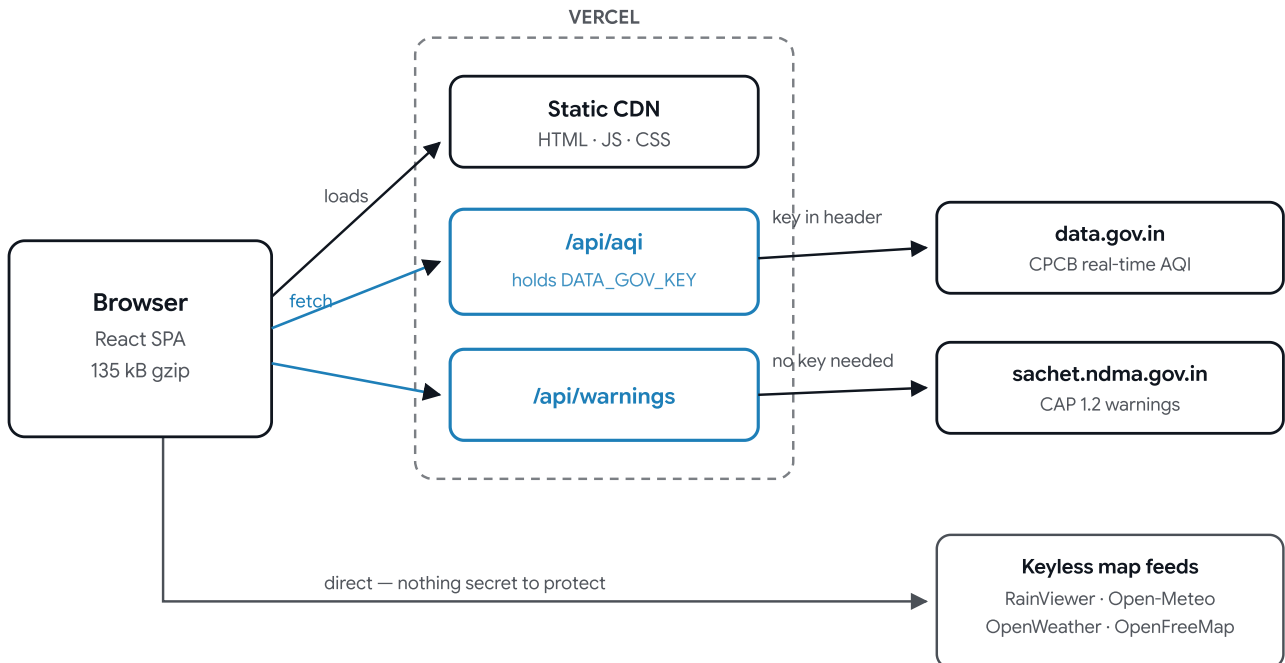
The eight personas, exactly as the problem statement defines them:

- **Health-conscious** — AQI, pollen, UV index, humidity
- **Outdoor fitness** — sunrise/sunset, best running hours, wind, heat alerts
- **Beachgoers & surfers** — sea state, tide timings, wave height, water temperature
- **Travellers** — saved destinations, flight-relevant alerts, packing suggestions
- **Parents & families** — school-commute conditions, rain and severe alerts
- **Farmers & gardeners** — soil moisture, rainfall, frost alerts, planting guidance
- **Commuters** — visibility, fog and storm alerts, road-relevant conditions
- **Event planners** — rain probability and a comfort index

Two things the brief does not say out loud but the app's own store reviews do: the current app is slow, and its favourites are unreliable. That set the second bar — the homepage had to be dramatically lighter than what ships today.

How the app actually works

A React single-page app served as static files, two serverless functions that hold the secrets, and four keyless feeds the browser is allowed to call directly.



The only arrow that carries a secret is the one from a serverless function to `data.gov.in`. The map feeds bypass the backend entirely because they need no key, so routing them through a function would add a hop and buy nothing.

What happens on load, in order

- Vercel's CDN serves `index.html` and one JS bundle. MapLibre is not in it.
- The app renders immediately from `data/seed.ts` — six cities of demo data. Nothing is blank while the network works.
- `useLiveAqi` calls `/api/aqi?city=...` for the selected city. `useLiveWarnings` calls `/api/warnings` once.
- When either answers, the value is merged into the place object in `AppState`. Cards, the banner, the advice line and the scoring rules all read the merged place, so nothing else needs wiring.
- Opening Places lazy-loads the MapLibre chunk. Only then does the browser pay 218 kB for a map.

The one design decision worth defending here: the seeded dataset is not a placeholder waiting to be deleted. It is the floor. If `data.gov.in` is slow or down during the demo, the card keeps the seeded number and changes its label from **LIVE** to **SEEDED**. The app never shows a blank card or a spinner that never resolves.

The stack, and why each piece

Four runtime dependencies. Everything else is a build tool that ships no code to the user.

React 18.3.1

GENERALLY

The most widely used UI library for building interfaces out of components that re-render when their data changes.

HERE

Every screen, card and sheet. State lives in one context, `AppState`, which merges live data into the seeded places.

WHY NOT VUE OR SVELTE

Both are fine choices. React won on hiring reality — it is what the team already knew, and a hackathon is not the place to learn a framework. Svelte would have produced a smaller bundle.

TypeScript 5.6

GENERALLY

JavaScript with a type checker. Catches mismatched data shapes before the code runs.

HERE

The `Place` type is the contract between seeded data, the CPCB response and every card. Adding a field without deciding where it comes from is a compile error.

WHY IT EARNS ITS COST

The provenance map is exhaustive by type. Forgetting to say whether a reading is live or seeded fails the build rather than shipping a wrong label.

Vite 5.4

GENERALLY

A build tool and dev server. Serves source files natively during development, bundles with Rollup for production.

HERE

Dev server, production bundle, code-splitting the map into its own chunk, and a small custom plugin that runs the `/api` routes locally.

WHY NOT CREATE REACT APP

CRA is deprecated and its dev server is markedly slower. Vite starts in under a second and its Rollup output is what gives us the lazy map chunk.

Tailwind CSS 3.4

GENERALLY

Utility classes for styling in markup, with unused classes stripped at build time.

HERE

Layout and spacing only. Every colour, blur, radius and spring comes from `design/tokens.ts` instead — one source of truth the build verifies.

WHY THE SPLIT

Contrast has to be computed against real gradients. That is arithmetic, and arithmetic belongs in TypeScript where a script can check it, not in class names.

Framer Motion 11

GENERALLY

An animation library for React with spring physics and automatic layout transitions.

HERE

Card enter/exit, FLIP reordering when the score changes, the pull-to-refresh gesture, and the detail sheet.

WHY NOT CSS TRANSITIONS

CSS cannot animate an element between two layout positions it does not yet occupy. Reordering the stack with a spring is exactly that problem, and `layout` solves it in one prop.

MapLibre GL JS 4.7

GENERALLY

An open-source vector map renderer. The community fork of Mapbox GL JS from before Mapbox changed its licence.

HERE

The map screen: radar tiles, wind tiles, and our own AQI labels drawn over an OpenFreeMap basemap.

WHY NOT MAPBOX OR GOOGLE MAPS

Both need an account, a billed key and a usage cap. MapLibre plus OpenFreeMap needs no key at all, which matters for a government app that should not depend on a commercial quota.

What is deliberately *not* here

COMMONLY EXPECTED	STATUS	WHY
Express	NOT USED	Vercel functions are plain <code>(req, res)</code> handlers. Express would add a dependency to do routing the platform already does.
TanStack Query	NOT USED	Two endpoints with different cache rules. Two small hooks are ~90 lines total; the library is larger than the problem.
SQLite	NOT USED	Serverless functions have no persistent disk. Caching is an in-memory <code>Map</code> per warm instance plus HTTP cache headers.
Redux / Zustand	NOT USED	One context with one state object. There is no cross-screen state complex enough to need a store.
An <code>axios</code> -style HTTP client	NOT USED	Global <code>fetch</code> is in Node 18+ and every target browser.

The serverless functions have **zero dependencies**. They import nothing but Node's own `http` types. That is why a cold start is fast and why there is no supply chain to audit on the server side.

Hosting, and where the backend went

Vercel

Generally: a hosting platform for front-end apps. It builds from a Git push, serves the result from a CDN, and turns files in an `api/` directory into serverless functions — small programs that run on demand and shut down after.

Here: it does both jobs. The Vite build output is served statically, and `client/api/aqi.ts` and `client/api/warnings.ts` become HTTP endpoints automatically. There is no `vercel.json` — the layout is detected.

WHY VERCEL AND NOT NETLIFY, RENDER OR A VPS

- **Netlify** is genuinely comparable and would have worked. Vercel's zero-config detection of a Vite app plus an `api/` folder needed no configuration file at all, which is one less thing to be wrong on demo day.
- **Render or a VPS** would give a long-running server and a static IP — see below. It also means managing a machine, and a free-tier Render instance sleeps and cold-starts in tens of seconds, which is a bad property for a live demo.
- **GitHub Pages** was ruled out immediately: static only, so there is nowhere to hide the CPCB key.

The honest trade-off a judge may find: IMD's own API requires the calling server's IP to be whitelisted. Vercel functions do not have a fixed outbound IP, so this hosting choice rules out ever calling `api.imd.gov.in` directly. We accepted that because NDMA's Sachet feed republishes the same IMD bulletins with no key and no whitelist. If IMD access is granted for the final build, the backend moves to a fixed-IP host and the two route files move with it unchanged — they are plain Node handlers, not Vercel-specific code.

Why the secret has to sit server-side

Anything Vite inlines into the bundle is public — a user can read it in devtools. Vite only inlines variables prefixed `VITE_`, so the split is enforced by naming:

VARIABLE	WHERE IT LIVES	WHY
DATA_GOV_KEY	Server only	Registered to us. In the bundle it could be lifted and used against our quota.
VITE_OWM_KEY	In the bundle	OpenWeather map tiles are fetched by the browser and the key is visible in the tile URL by design. Free tier, rate-limited by them.

The same `/api` files run in development through a custom Vite plugin (`vite-plugin-api.ts`) that imports them via Vite's SSR loader. It is the same TypeScript file Vercel builds — not a second copy — so a route that works locally works deployed.

Which numbers are real

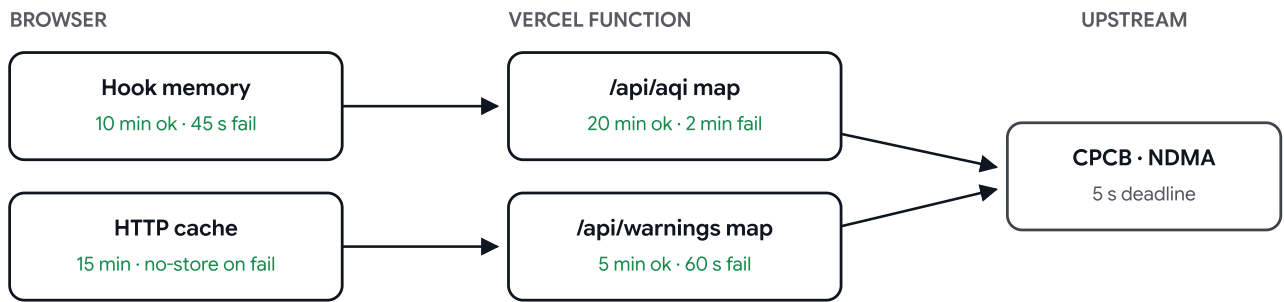
This is the section to know cold. Every value on screen carries a **LIVE** or **SEEDED** badge in the app itself, so a judge can check any claim by tapping it.

DATA	STATUS	SOURCE
Air quality index, PM2.5, PM10	LIVE	CPCB continuous monitoring via data.gov.in
Severe weather warnings	LIVE	NDMA Sachet CAP feed — carries IMD, SDMA and CWC bulletins
Precipitation radar	LIVE	RainViewer
Wind overlay	LIVE	OpenWeather Weather Maps
Map air-quality labels	LIVE	Open-Meteo — US AQI , not CPCB
Temperature, humidity, wind, visibility	SEEDED	data/seed.ts
Rain probability, hourly outlook	SEEDED	Derived from a seeded 24-hour curve
Soil moisture, Agromet advisory	SEEDED	Written to read like a real AAS bulletin, labelled as demo
Tides, wave height, sea temperature	SEEDED	No marine source is wired. INCOIS has never been called.
Urban waterlogging, named junctions	SEEDED	Illustrative — no urban nowcast source exists in this build
Airport / aviation rows	SEEDED	Illustrative, not IMD Aviation Services

Two AQI scales are on screen and that is deliberate. The saved-place cards show **CPCB Indian AQI**, computed by our own server from raw pollutant concentrations. The map shows **US AQI** from Open-Meteo, because it is the only keyless source with national coverage per coordinate. The two use different breakpoints, so the same city legitimately reads differently. The map chip says **US AQI** for exactly this reason. Do not let a judge catch this before you say it.

Caching, and why one TTL would be wrong

CPCB publishes hourly. CAP warnings arrive when they are issued, not on a schedule. Radar refreshes every few minutes. Using one number for all of them would either hammer the upstream or serve stale warnings.



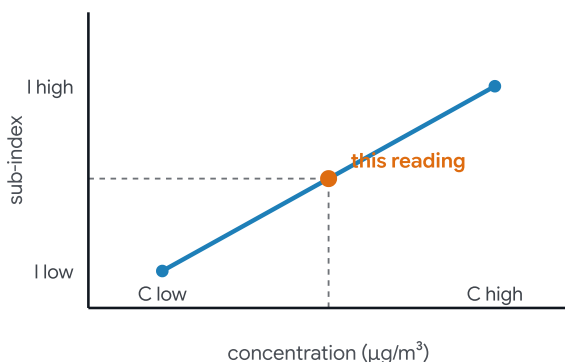
Every layer keeps a separate failure lifetime — a failed fetch must not be cached like a success.
Pull-to-refresh clears the hook cache and re-requests with cache: "reload".

Five stores, each with its own success and failure lifetime. The failure split was a real bug: a failed response was once cached for fifteen minutes, so `curl` returned live data while the app still showed "seeded".

The AQI maths, in full

data.gov.in publishes *pollutant concentrations*, not an index. The index is computed on our server, using CPCB's published method.

Six pollutants are used: PM2.5, PM10, NO₂, SO₂, NH₃ and ozone. Each has a table of breakpoints mapping a concentration band to an index band. For a reading inside a band, the sub-index is a straight-line interpolation:



SIX SUB-INDICES



AQI = **max** of the six

PM10 governs this station

The card names the governing pollutant, not just the number.

$$I = I_{\text{low}} + (I_{\text{high}} - I_{\text{low}}) / (C_{\text{high}} - C_{\text{low}}) \times (C - C_{\text{low}})$$

Indian AQI is defined as the worst sub-index, not an average. That is why the card shows which pollutant produced the figure — an AQI of 169 driven by NO₂ means something different from one driven by PM2.5.

DETAILS WORTH KNOWING

- **Carbon monoxide is deliberately excluded.** CPCB defines CO breakpoints in mg/m³ while the feed publishes µg/m³. Mixing the units would produce a wildly wrong sub-index, and since AQI is a maximum, one wrong sub-index corrupts the whole index.
- **Up to 600 rows per city.** Delhi returns roughly 308 rows across 44 stations. An earlier limit of 200 silently truncated it.
- **Station names are exact-match.** CPCB files the capital under "Delhi", not "New Delhi" — that mismatch returned zero rows and the card showed seeded data under a live label for the entire life of the feature. Each place now carries an explicit `cpcbCity`.
- **Kochi has no CPCB station at all.** Kerala reports Kannur, Thiruvananthapuram and Thrissur. The card says so rather than inventing a number.
- The index is capped at 500, as CPCB caps its published figure.

The warning pipeline

Severe weather is the one thing in this app that must never be wrong, and it is the one thing that is fully live.

CAP — the Common Alerting Protocol — is an OASIS standard for emergency alerts, used internationally. NDMA publishes India's alerts as CAP 1.2 XML through its Sachet portal, with no key and no whitelisting, and that feed carries the same IMD, state SDMA and Central Water Commission bulletins that IMD's own API would.

WHAT /API/WARNINGS DOES

- Fetches the national RSS index (6-second deadline), then fetches individual CAP entries through a **12-way worker pool under a 7-second budget** — enough parallelism to enrich the feed without exceeding a serverless invocation.
- Maps CAP's `severity` to IMD's public colour code, which happens to be the same four-value shape:

EXTREME → RED	SEVERE → ORANGE	MODERATE → YELLOW	MINOR → GREEN
---------------	-----------------	-------------------	---------------
- Drops anything not currently in force: `status` other than `Actual`, a `msgType` of `Cancel`, or an `expires` time in the past. An expired Chennai bulletin was on screen before this check existed.
- Matches districts against each saved place using **word-boundary Unicode matching**. Plain substring matching put a Tamil Nadu thunderstorm on Indore, because "Dhar" appears inside "Dharmapuri".

A live warning is never translated. The app has full Hindi, but a live CAP bulletin is shown exactly as issued. Rewording a real warning would put words in IMD's mouth. Seeded bulletins do carry Hindi, because we wrote them. The rule is applied in one place, `i18n/capText.ts`, and it covers the headline, the body, the expiry time and the issuing office.

How a card earns its place

The problem statement asks for personalization. Ours is arithmetic, not a model — a judge can ask why any card is where it is and get a specific answer with numbers in it.

score = base + urgency boost



On screen, in the reader's language:

"Commuters is persona 2 of 2, worth 90. In Mumbai right now, visibility is down to 2.4 km, below our own 3 km driving-caution line — that adds +36, for 126."

Every card carries a "Why this card" disclosure holding that sentence, assembled from the same numbers the rule used. Nothing is hidden behind a model.

Three rules the engine also applies

- **Severe warnings outrank everything.** The banner is pinned above the persona cards regardless of which personas are active.
- **Suppression is explained, not silent.** The beach card cannot show for an inland city, so it is removed and a line below the stack says which persona was suppressed and why.
- **Advice ordering is by consequence, not by data source.** Life-safety first, then whatever the active warning is about, then health, then comfort — capped at four items, because a list of nine things to do is a list nobody reads.

PACKING SUGGESTIONS REUSE THE SAME ENGINE

The problem statement asks travellers for packing suggestions — "carry a raincoat in London". Rather than write a second rule set, each destination's top advisory is projected onto a thing you can put in a bag: *carry an umbrella* becomes *pack rain cover*. Advisories with no honest packing equivalent — "use fog lights", "avoid the underpass" — are absent rather than stretched into one.

The design system

One TypeScript file, `design/tokens.ts`, holds every colour, blur, radius and spring in the app. Nothing is hard-coded in a component.

The sky is a lookup, not a gradient someone picked

The background is a function of two inputs: **condition** (clear, partly cloudy, overcast, rain, thunderstorm, fog, snow) × **time of day** (dawn, day, dusk, night). That is $7 \times 4 = 28$ defined gradients.

Time of day derives from the *location's* own sunrise and sunset via `Intl.DateTimeFormat`, not the device clock — so checking Kochi from Delhi shows Kochi's night. Each place carries an IANA timezone for this reason.

Glass, and the contrast problem it creates

Cards are frosted surfaces over that moving background. That is a legibility risk in a severe-weather app, so contrast is not eyeballed — it is computed. A build script composites each glass surface over the real gradient at the screen position where cards actually sit, then measures the WCAG contrast ratio of both text colours against it.

CHECK	RESULT
Skies clearing WCAG AA 4.5:1 on glass and bare	28 / 28
Skies needing a scrim behind the hero text	16
Hourly-chip tint samples measured	700
Worst chip contrast found	5.02 : 1

Two glass tiers exist — one for dark skies, one for light — and the app measures which polarity a given sky needs rather than assuming. There is also a flat, blur-free tier for low-end Android, exposed as a toggle, because `backdrop-filter` is the single most expensive thing on the screen.

Performance

The current app's reviews complain about slow loads. This is the number that answers that.

ASSET	GZIPPED	WHEN
Main JS bundle	135.1 kB	First load
CSS	5.4 kB	First load
HTML	0.7 kB	First load
MapLibre GL	217.6 kB	Only when the map opens
MapLibre CSS	9.2 kB	Only when the map opens
Devanagari font	121.2 kB	Only when a Hindi glyph renders

Quote the number carefully. "135 kB" is the JavaScript bundle. A cold first visit in English is closer to **250 kB** once the web fonts are counted. Both are good figures for a weather app — but say which one you mean, because a judge with devtools open will see the larger number.

WHERE THE SPEED COMES FROM

- **The map is deferred.** 218 kB of MapLibre is a separate chunk that only downloads when someone opens Places. That is the single biggest win, and it is a deliberate architectural choice rather than a small bundle by luck.
- **Blur is dropped while the stack animates.** A transformed `backdrop-filter` element re-blurs its backdrop every frame; across a stack of cards that is the dominant cost. A short motion window swaps in a flat surface during reorders.
- **Fonts are subset by unicode-range.** An English session downloads the Devanagari stylesheet but none of its 121 kB of glyphs.
- **Skeletons, never spinners.** Every fetch has a defined loading and offline state.

Two languages, verified

English and Hindi are complete: **444 keys in each**, including every seeded advisory, city name, station name, AQI band, soil category and warning bulletin. The language picker also lists ten more Indian languages as downloadable packs — that list is a UI demonstration, and only English and Hindi are implemented.

The hard part was not the UI chrome, it was the data. Seeded prose used to render straight from the data file in English, so a Hindi sentence would switch language halfway through. Those strings now live in the dictionary with the data file's English as the fallback of record, and the build diffs the two copies.

What stays in English on purpose: live CPCB station names and live CAP bulletins, shown exactly as the upstream issued them. Plus acronyms and codes — IMD, CPCB, AQI, IST, RMC, state codes.

What the build refuses to ship

`npm run check` runs the type checker plus eight verification passes. These exist because each one corresponds to a bug that actually happened.

#	CHECK	THE BUG IT PREVENTS
1–3	Design tokens match the design study	The standalone prototype drifting from the app
4	Contrast on all 28 skies, both text contexts	Unreadable text on one weather condition nobody demoed
5	Contrast on 700 hourly-chip tint samples	The tinted carousel dropping below AA at one hour
6	Both dictionaries in step; every literal <code>t()</code> key resolves	A key that exists in English and not Hindi
7	CAP English in the dictionary matches the data file	Rewording a bulletin and leaving the Hindi behind
7b	All 107 seeded strings have a translation and have not drifted	Exactly the half-English sentence described above
8	Serverless routes have no relative imports	A real production crash — see below

Check 8 has a story worth telling. `/api/aqi` imported a sibling file, which works locally but produced `ERR_MODULE_NOT_FOUND` on Vercel, because `"type": "module"` means emitted ESM and ESM cannot resolve an extensionless relative import. The fix was to inline the shared code so each route is self-contained; the check makes sure nobody reintroduces it. It was tested by putting the bug back and confirming the build failed.

Honest limits

Say these before a judge finds them. Every one has a reason and a route forward.

Is this a PWA? Can I install it?

Not yet. There is no web app manifest and no service worker. The original plan called for both; neither is built. It is a responsive web app that works well on a phone browser.

Does it work offline?

No — and be careful with the wording here. The seeded dataset means the app never shows a blank screen when a *feed* fails, and there is a defined stale-data state with an "updated X ago" timestamp. But with no network at all, the page itself will not load, because there is no service worker caching it. Demo-safe is not the same as offline-first.

Are my saved places and personas remembered?

Within a session, yes. Across a reload, no — nothing is persisted to `localStorage` yet. Given that broken favourites are a top complaint about the current app, this is the first thing to build next; it is roughly an hour of work.

Why are the forecast numbers not live?

IMD's forecast APIs require the calling server's IP to be whitelisted, which is a procurement timeline, not an engineering one. We wired everything that *is* reachable without it — CPCB air quality and NDMA's CAP warnings — and labelled everything else as seeded rather than presenting demo values as measurements.

Is there any traffic data on the commute card?

No, and the card does not imply one. It reads visibility, gusts and waterlogging risk — weather-side commute conditions. Traffic integration is a named next step, not a claim.

Pull-to-refresh – what actually refreshes?

Air quality does: the gesture clears the client cache and re-requests with `cache: "reload"`. Warnings do not currently invalidate on pull; they refresh on their own five-minute cycle. That is a known gap.

Is the "updated X ago" timestamp real?

The live air-quality card shows CPCB's own publication timestamp, which is real. The header's overall data age is currently a fixed value, not a measured one.

Rapid fire

One-line answers to the questions most likely to come fast.

Why not just use OpenWeather for everything?

The brief is an IMD product. A homepage for India's national meteorological agency that quietly runs on an American commercial API would fail on the thing it is being judged for. We use OpenWeather for one map overlay and say so.

How is this personalized if there is no login?

Persona choice plus location plus time of day. No account, no tracking, no profile leaving the device — which is also why it needs no privacy policy to demo.

Why rule-based instead of machine learning?

Explainability is a hard requirement, not a preference. A judge asking "why is this card here" gets a sentence with the threshold and the reading in it. An ML ranker could not answer that, and there is no training data for eight personas anyway.

How many API calls does a screen load make?

Two to our backend — air quality for the selected city, and warnings once for all saved places. The map adds its own requests only when opened.

What happens if data.gov.in is down during this demo?

A five-second deadline and one four-second retry, then the card keeps its seeded value and relabels itself as seeded. You will see the badge change; nothing breaks.

How do you know the Hindi is complete?

The build fails if any key exists in one language and not the other, and separately if any of the 107 seeded strings loses its translation. It is not a manual check.

Could this scale to all of India?

The map's air layer already covers 154 cities in three zoom tiers. The persona and card logic is per-place and stateless, so scaling is a matter of the data source's coverage, not of our architecture.

What is the single most technically interesting thing here?

Pick one and own it. Reasonable answers: computing CPCB's Indian AQI from raw concentrations on our own server; the zoom-tiered map labels that prevent clutter by never drawing it; or a build that refuses to ship a contrast failure or a missing translation.

Written from the codebase at commit 011b411 · 18 commits · ~11,000 lines TS/TSX/CSS · verified against package.json, client/api, and src/design/tokens.ts